

Trabajo Práctico Final

Parte A

I-402 - Principios de la Robótica Autónoma

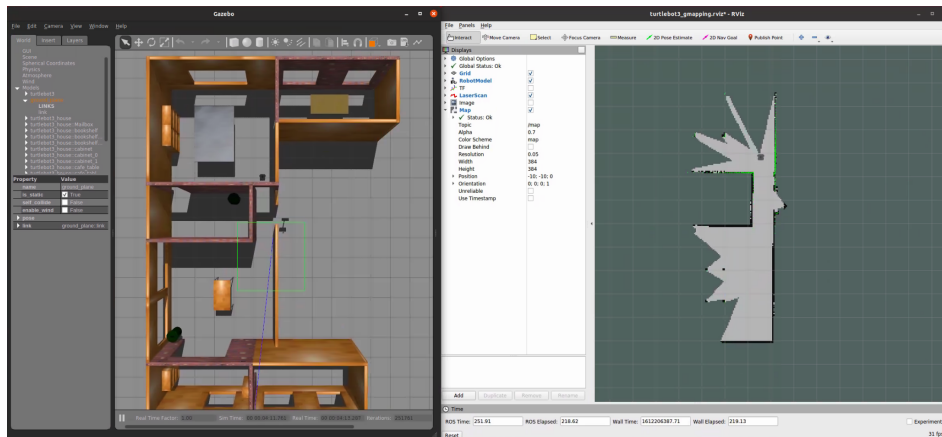
Prof. Ignacio Mas, Tadeo Casiraghi y Bautista Chasco

En este trabajo práctico final, los alumnos deberán integrar los principales conceptos de los enfoques probabilísticos abordados a lo largo de la materia mediante la implementación de un sistema autónomo de localización y mapeo simultáneos (SLAM).

Utilizando un robot TurtleBot3 simulado en el entorno de Gazebo, o bien un RosBag del TurtleBot4, el objetivo en esta primera etapa (Parte A) será que el robot explore un entorno desconocido de tipo laberinto y construya una representación (mapa) del mismo mientras estima su propia posición.

Esta fase representa un caso de aplicación realista donde convergen múltiples herramientas estudiadas durante la cursada, tales como la estimación de estado en presencia de ruido, el procesamiento de sensores inexactos (como LIDAR o cámaras) y la fusión de información sensorial. La correcta implementación de esta etapa es fundamental, dado que el mapa generado y la precisión de la localización serán el pilar sobre el cual se desarrollarán las Partes B y C, centradas en la navegación autónoma.

SLAM - Generación del mapa



En esta primera etapa, los alumnos deberán implementar un sistema de SLAM utilizando el robot TurtleBot3 en un entorno simulado en Gazebo o el TurtleBot4 con datos pregrabados. El objetivo principal es que la plataforma explore un escenario tipo laberinto y construya un mapa de ocupación empleando las lecturas de los sensores y la odometría.

Esta etapa permite aplicar de forma práctica los algoritmos de estimación de estado y mapeo estudiados en la materia, ya sea mediante técnicas de SLAM basadas en Filtros de Partículas (FastSLAM), Filtros de Kalman Extendidos (EKF SLAM) o cualquier otro método pertinente. Al finalizar esta sección, se deberá exportar el mapa generado y verificar su calidad, ya que servirá como base fundamental para la navegación en la siguiente fase del trabajo.

Para el desarrollo de esta primera parte, deberán elegir uno de los 3 caminos posibles de implementación.

1. Opción 1: Grid-Based FastSLAM

Entorno de Trabajo	Gazebo
Sensores	LIDAR
Algoritmos	Likelihood Fields Modelo de Odometría Filtro de Partículas Occupancy Grids Grid-Based FastSLAM

Para este camino deberán implementar el algoritmo Grid-Based FastSLAM en el entorno simulado de Gazebo. Se verán forzados a optimizar el código para que no solo se ejecute de manera eficiente en sus equipos, sino que también permita procesar una cantidad razonable de partículas en tiempo real. Para cada partícula, deberán mantener un mapa estimado mediante *likelihood fields* para lograr una localización efectiva mientras realizan el mapeo.

1.1. Preparación del entorno

Antes de comenzar con la implementación de SLAM, es necesario configurar correctamente el entorno de simulación. Para esto, se debe contar con una instalación funcional de ROS 2 Humble, Gazebo y los paquetes de TurtleBot3, ya sea de forma nativa en Ubuntu 22.04 (o cualquier sistema operativo compatible) o mediante Robostack. Recuerden que disponen de las computadoras del laboratorio de informática en el edificio Sullair; no obstante, si optan por esta alternativa, actúen con cautela para no dejar el código desarrollado expuesto en equipos públicos.

1.2. Lanzamiento del robot y teleoperación

Una vez configurado el entorno, el siguiente paso es lanzar el robot TurtleBot3 dentro del laberinto simulado para explorar el terreno. Para ello, ejecuten los siguientes comandos en dos terminales independientes:

- **Terminal 1:** `ros2 launch turtlebot3_custom_simulation custom_casa.launch.py`
- **Terminal 2:** `ros2 run turtlebot3_teleop teleop_keyboard`

1.3. Odometría

La odometría requerida es la publicada en el tópico `calc_odom`. Esta representa la posición estimada del robot, por lo que deben contemplar que acumulará un error considerable a lo largo del tiempo. Si buscan actualizar la odometría utilizando el modelo de deltas ($\delta_{\theta_1}, \delta_{\theta_2}, \delta_{trans}$), recuerden calcular la diferencia respecto a la odometría del instante de tiempo anterior.

1.4. SLAM en ROS 2

En esta etapa, diseñarán e implementarán su propio algoritmo de SLAM utilizando las lecturas del TurtleBot3. El propósito es construir el mapa del entorno al mismo tiempo que se estima la pose del robot. Al finalizar, guarden obligatoriamente este mapa, ya que será el insumo de localización para las partes B y C del TP Final.

1.5. Visualización con RViz

Durante la ejecución del algoritmo, se recomienda fehacientemente el uso de RViz para monitorear las señales de entrada y la salida del sistema. Los elementos clave a visualizar son:

- `/scan`: Visualización de los rayos del LIDAR.
- `/odom`: Posición real del robot (*ground truth*).
- `/calc_odom`: Posición estimada por odometría pura.
- `/belief`: Pose corregida del robot (estimación de SLAM).
- `/map` o `/likelihoodfield`: Mapa de ocupación generado en tiempo real.

Pueden cargar un archivo `.rviz` preconfigurado o añadir los elementos manualmente en la interfaz.

1.6. Mapa final

El entregable final de esta opción consistirá en un mapa de grilla de ocupación optimizado, el cual será exportado formalmente para su uso posterior en las etapas de navegación.

2. Opción 2: Features con LIDAR

Entorno de Trabajo	Gazebo
Sensores	LIDAR
Algoritmos	Extracción de características (features) mediante LIDAR Modelo de Odometría Algoritmos de SLAM basados en características, p. ej.: - EKF SLAM - Graph SLAM - SEIF SLAM

Para este camino, deberán implementar un módulo de detección y extracción de características geométricas (*features*) a partir de los datos crudos del LIDAR. Tengan en cuenta que estos puntos de referencia deben ser razonablemente estables y repetibles ante cambios de perspectiva para actuar como hitos (*landmarks*) confiables.

Una vez resuelta la extracción, podrán aplicar el algoritmo de SLAM basado en características de su preferencia. En este caso, el proceso de mapeo es doble:

1. **Primera pasada:** Construcción y consolidación del mapa de características puntuales para resolver la localización del robot.
2. **Segunda pasada:** Con la localización ya resuelta y corregida por el mapa de características, realizarán un mapeo de grilla de ocupación clásico con el LIDAR. Esto simplifica el proceso, ya que no deberán preocuparse por la distorsión del mapa derivada del error de trayectoria.

Este mapa de grillas definitivo es estrictamente necesario para que el robot pueda planificar trayectorias fluidas sin colisionar con los obstáculos en las Partes B y C.

2.1. Preparación del entorno

El entorno operativo replica las especificaciones de la sección anterior: requiere ROS 2 Humble, Gazebo y las dependencias de TurtleBot3 configuradas en sus entornos locales o en las terminales asignadas en el laboratorio Sullair.

2.2. Lanzamiento del robot y teleoperación

El despliegue de la simulación y el nodo de teleoperación manual se efectúa mediante los mismos comandos de lanzamiento de la opción previa.

2.3. Odometría

Al igual que en la opción anterior, se deberá consumir el tópico `calc_odom` y procesar de forma adecuada el modelo de deltas cinemáticos para la actualización predictiva del filtro o grafo elegido.

2.4. SLAM en ROS 2

Diseñen el nodo de SLAM para estimar simultáneamente las posiciones del robot y las coordenadas de los hitos detectados. Al concluir, el sistema debe ser capaz de guardar las estructuras de datos correspondientes a la infraestructura del mapa.

2.5. Visualización con RViz

Monitoreen el desempeño del estimador en RViz prestando especial atención a:

- `/scan`: Puntos del LIDAR e hitos detectados superpuestos.
- `/odom` y `/calc_odom`: Comparativa de trayectorias.
- `/belief`: Pose corregida bajo el mapa de características.
- `/landmarks` o `/map`: Representación de los hitos y la grilla de ocupación según la pasada actual.

2.6. Mapa final

El resultado esperado obligatorio para esta opción consta del mapa de grilla de ocupación del entorno y, de manera complementaria, el mapa geométrico con la ubicación de las características (landmarks) validadas.

3. Opción 3: Features con Cámara

Entorno de Trabajo	RosBag (Datos reales de TurtleBot4)
Sensores	LIDAR, Cámara
Algoritmos	Detección y extracción de características vía ArUco Tags Modelo de Odometría Graph SLAM (Obligatorio)

Debido a que el entorno estándar de Gazebo no modela con suficiente fidelidad los efectos ópticos de una cámara real, para esta opción trabajarán con un archivo de registro de datos (*RosBag*). Un *RosBag* es una grabación temporal de los tópicos y mensajes del sistema; al reproducirlo, sus nodos locales recibirán la información de la cámara y el LIDAR exactamente como si la plataforma física estuviera operando en vivo.

Deberán emplear este registro para diseñar un algoritmo robusto de detección de marcadores ArUco. Esto implica mitigar problemas reales como el desenfoque por movimiento (*motion blur*) y la baja densidad de los tags en ciertas zonas del laberinto. Para esta opción, **es obligatorio implementar Graph SLAM**, incorporando mecanismos de cierre de lazo (*loop closure*) para optimizar el grafo global de poses.

Al igual que en la opción anterior, el procedimiento requiere dos etapas: una primera aproximación para consolidar el grafo de poses y landmarks visuales, y una segunda reproducción con la trayectoria corregida para proyectar las lecturas de LIDAR en una grilla de ocupación consistente.

3.1. Preparación del entorno

En este escenario no se ejecutará Gazebo. Deberán descargar los paquetes de datos, la matriz de calibración intrínseca de la cámara (K) y los coeficientes de la cámara desde el siguiente ENLACE. Los archivos incluidos son:

- **aruco_estimation.zip**: Grabación corta con un marcador ArUco dispuesto a diferentes distancias controladas, ideal para calibrar y caracterizar el modelo de medición.
- **laberinto.zip**: Registro extenso que recorre el laberinto completo realizando múltiples bucles. Se utilizará para la generación del mapa final.
- **Matrices, coeficientes y estimaciones**: Archivo con los datos de las cámaras utilizadas. El RosBag fue grabado con el Turtlebot 4 número 0.

3.2. Lanzamiento del robot y teleoperación

Para iniciar la reproducción de los datos grabados, utilicen el comando nativo de ROS 2:

```
ros2 bag play nombre_de_la_carpeta_del_bag
```

Al ejecutarse, los tópicos comenzarán a publicar de forma transparente para sus nodos de procesamiento.

3.3. Odometría

La odometría a utilizar se publica bajo el tópico `tb4_0/odom`. Al provenir de un entorno real, posee un nivel de ruido y deriva acumulada característico de los sensores físicos. Apliquen las diferencias temporales discretas para extraer las transformaciones relativas ($\delta_{\theta_1}, \delta_{\theta_2}, \delta_{trans}$).

3.4. SLAM en ROS 2

El objetivo es estructurar el algoritmo de Graph SLAM para estimar la trayectoria basándose en las restricciones relativas inducidas por la odometría y las visualizaciones de los ArUco Tags. Noten que en este caso práctico no se dispone de *ground truth* (posición real). Es crucial estructurar un buen criterio de cierre de lazo para corregir la deriva acumulada al regresar a puntos previamente visitados.

3.5. Visualización con RViz

Para validar el comportamiento del estimador basado en grafos, configuren RViz visualizando los siguientes elementos:

- `tb4_0/scan`: Nube de puntos proveniente del LIDAR real.
- `tb4_0/odom`: Trayectoria estimada originalmente por odometría.
- `/belief`: Trayectoria corregida tras la optimización del grafo.
- `/landmarks`: Posiciones estimadas de los marcadores ArUco en el mapa.
- `/poses_guardadas`: Visualización de los nodos que componen el grafo de optimización.
- `/map`: Grilla de ocupación resultante generada a partir de la trayectoria corregida.

Se sugiere además abrir una ventana de visualización de imágenes con las detecciones de los tags dibujadas sobre el video para certificar la estabilidad del detector visual.

3.6. Mapa final

El entregable indispensable constará del mapa en formato de grilla de ocupación (métrica) y el archivo con las localizaciones relativas de los hitos visuales identificados por sus respectivos IDs.

4. Evaluación del mapa

Para determinar la calidad, precisión técnica y la correcta implementación del algoritmo de SLAM seleccionado, el cuerpo docente evaluará rigurosamente los siguientes tópicos:

4.1. Coherencia con el entorno real o simulado

El mapa final debe reflejar de manera fidedigna la geometría del entorno físico o virtual propuesto. Las paredes, esquinas y pasillos deben estar definidos nítidamente. No se admitirán distorsiones geométricas severas, solapamientos extraños de muros ni la presencia de obstáculos fantasma (ruido artificial) o la omisión de paredes reales. El entorno debe ser explorado y mapeado en su totalidad.

4.2. Resolución y nivel de detalle

La resolución espacial elegida para la grilla o la precisión en la convergencia de los hitos geométricos debe ser suficiente para discernir detalles críticos (aperturas de puertas, giros cerrados, pasajes estrechos). El mapa debe estar libre de ruido disperso y no presentar discontinuidades o saltos abruptos que inhabiliten su posterior uso en algoritmos de planificación.

4.3. Consistencia temporal

El mapa debe permanecer estable a medida que transcurre el tiempo y el robot vuelve a pasar por zonas conocidas; las actualizaciones consecutivas no deben provocar fluctuaciones erráticas ni divergencias en las estructuras ya consolidadas. El estimador debe mantener un rastreo coherente de la pose sin perderse o romper la topología del entorno durante la exploración.

4.4. Uso para navegación

El criterio definitivo de aceptación del mapa es su viabilidad operativa. La grilla generada debe ser apta para que un planificador de caminos global (como A* o Dijkstra) pueda trazar rutas seguras entre coordenadas arbitrarias del laberinto. Se evaluará la capacidad del robot para localizarse con precisión matemática y trasladarse con éxito sobre el mapa diseñado por los alumnos.

5. Entregables

Se solicita adjuntar los paquetes de ROS creados, incluyendo archivos de lanzamiento (*launch files*) configurados y documentación clara para su ejecución en la defensa del trabajo.